

BTW Sandpile Model Report

Ruth Roberts

The Art of Scientific Communication

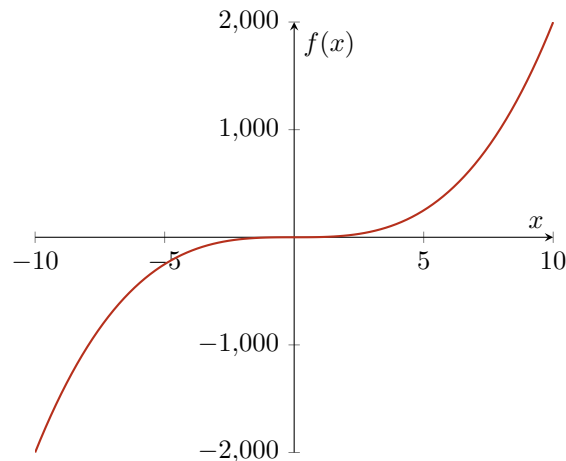
1 Introduction: The Sandpile Model

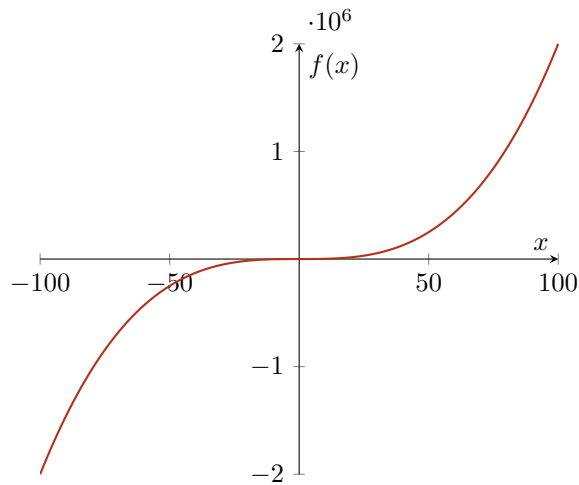
1.1 Self Organised Criticality and Scale Invariance

The Sandpile model was first introduced by Bak, Tang and Wiesenfeld [1] as a basic example of what they called self organised criticality, a property of systems with large degrees of freedom characterised by scale invariance, meaning the "shape" of the system is the same regardless of the length scale chosen. The Bak, Tang and Wiesenfeld (BTW) Sandpile model was introduced in order to explain $\frac{1}{f}$ noise present in many physical phenomena, including resistors and stars. In mathematics, a function is described as scale invariant if the shape of the function remains essentially the same whether the scale is small or large. These functions are typically characterised by power laws of the form

$$f(x) = ax^b.$$

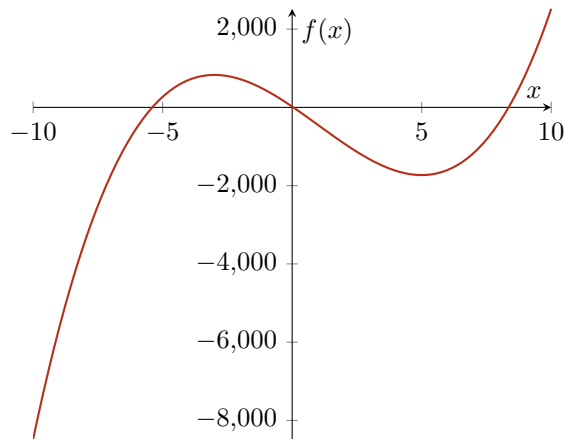
This is demonstrated by the following plots of $f(x) = 2x^3$, on different scales:

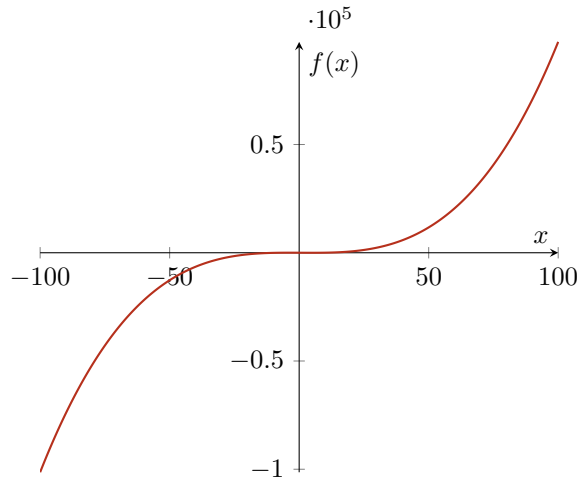




The two functions look exactly the same, even though they are being viewed from much different length scales. Such scale invariant functions are typically power laws, because scaling the x values by some constant a results in the y values all being scaled by a^3 . For example when we "zoom out" in the function $f(x) = 2x^3$, as in the above plots, each x value in the first plot is in the same position as $10x$ in the second plot, and therefore each y value in the first plot is in the same position as 10^3y in the second plot, ensuring that the second graph retains the shape of the first. The preservation of shape would not be possible for a polynomial of the form $ax^3 + bx^2 + cx + d$, since multiplying x by a (or any number) would not give the same proportionality in y in this case, instead each y term would be scaled by a different value. This is demonstrated by the plots of $g(x) = 2x^3 - 5x^2 + 3x + 2$

$$(x - 5)(x + 3) \quad x^2 - 2x - 15 \quad \frac{1}{3}x^3 - x^2 - 15x$$

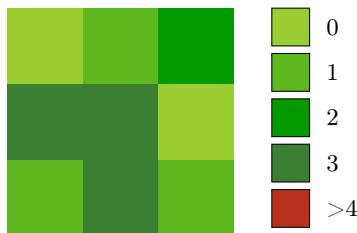




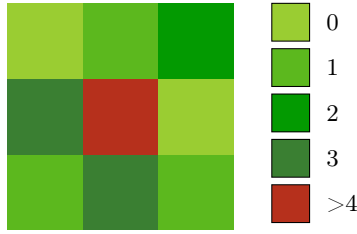
The extreme points are far more obvious in first plot compared to the second, causing the shape of the plot to appear completely different on a different length scale because this function is not scale invariant.

1.2 The Sandpile Model

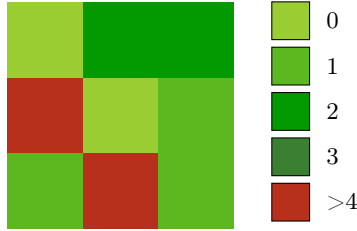
The sandpile model models piles of grains of sand on a square two-dimensional lattice. A grain of sand is added to a site chosen uniformly at random each timestep, and there is some threshold value (usually taken to be 4 on a two-dimensional lattice) at which a pile becomes unstable and "topples", distributing its grains of sand to neighbouring piles. This can cause the neighbouring piles to topple, causing an "avalanche" which, with large grids, can consist of hundreds or even thousands of topples. For example, consider the following stable 3×3 lattice, where darker green corresponds to more grains of sand in the pile:



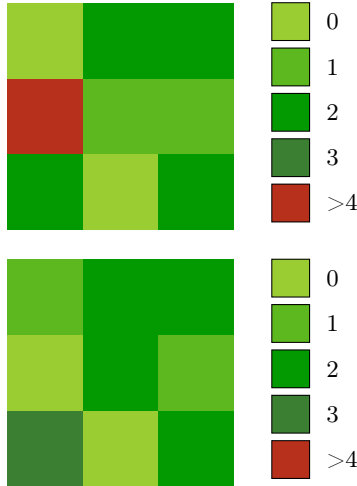
Now imagine a grain of sand is dropped on the centre of the lattice, taking it above the threshold (represented by the red square):



The pile then topples and redistributes grains to its neighbours:



Now we see that two more piles need toppling. Because of the Abelian property of the Sandpile Model (which will be discussed in more detail in the next section), regardless of which site we topple first, the final configuration of the lattice will be the same. We continue toppling until the lattice is stable, i.e. all values are below the threshold.



This is a simple illustrative example, but it is easy to see how the dynamics of this model can get quite complicated on a large lattice as there are many degrees of freedom due to the amount of individual piles of sand. This is where the self organised criticality of the system comes from.

1.3 The Abelian Property

The Abelian property is fundamental to understanding and simulating BTW Sandpile models. Informally, an operation has the Abelian property if, when applied multiple times, the order of application of the operations. For example, the basic operations addition and multiplication on the set of real numbers have the Abelian property while subtraction and division do not. To show how the toppling operator in the BTW model satisfies this property, I will define the toppling operator mathematically and explain how the order of topples does not impact the end result.

Consider an $n \times n$ lattice L and a site on the lattice (i, j) . We represent the lattice as a matrix with $l^{(ij)}$ denoting the number of grains of sand in the pile at site (i, j) . Let T_{ij} be the toppling operator for site (i, j) . Then we can define a matrix Δ_{ij} such that $T_{ij}(L, (i, j)) = L + \Delta_{ij}$. Then, for a d dimensional lattice, we have:

$$\delta_{ij}^{(i'j')} = \begin{cases} -4, & \text{if } i' = i, j' = j \\ 1, & \text{if } i' = i \pm 1 \text{ and/or } j' = j \pm 1 \\ 0, & \text{otherwise.} \end{cases}$$

Hence, if we take multiple sites $(i_1, j_1), (i_2, j_2), (i_3, j_3)$ in a lattice L , toppling these sites is the same as adding $\Delta_{i_1 j_1}, \Delta_{i_2 j_2}$ and $\Delta_{i_3 j_3}$ to the lattice L , which is essentially addition in $\mathbb{R}^n$ so is commutative. Hence, the model is Abelian as long as the order of toppling sites does not affect the sites that will be toppled. This is clearly the case because the only way for an unstable site to become stable again is by being toppled, meaning once a site becomes unstable, it cannot become stable again unless it is acted upon by the toppling operator.

1.4 Simulating the BTW Model

In this project I will simulate a BTW model on a various square lattices (side lengths 80, 90, 100, 110 and 120) over 500,000 timesteps. Simulating the lattice must be done with finite boundaries because the computer cannot store and operate on an infinite array. Furthermore, the number of different possible configurations of pile sizes on an infinite lattice would not be countable. I will also record and analyse the following statistics for each avalanche:

Area: the number of sites toppled in the avalanche.

Length (radius): The Manhattan distance between the first site toppled (the site which received the dropped sand) and the furthest site whose pile size has been affected in the avalanche. This could be a site that is toppled or the neighbour of a site that has been toppled meaning the pile at the site has increased in size without needing to be toppled.

Topples: The number of topples occurring in the avalanche.

Loss: The number of grains of sand lost at the boundary of the lattice during the avalanche.

Mass: The number of grains of sand in the whole grid

1.5 Predictions

Because grains of sand are added to the model at random and can be lost, I do not expect the system to reach a steady state, where properties of avalanches or statistics of the lattice itself are constant over time. For the same reason, I do not expect the number of grains of sand in the lattice to reach an equilibrium state. However, I would expect the system to reach a statistically stationary state, where the statistical distributions of properties of avalanches and the lattice itself do not change over time. I would also expect the average number of grains of sand in the lattice and grains of sand in each site to remain constant; once there are enough grains of sand (two or three grains at most sites in the interior of the lattice), any site in the interior that is toppled is fairly likely to have its pile replenished by the avalanches of its neighbours. This was the case in the example above, where the central site finished the avalanche with two grains of sand in its pile due to the avalanches of two of its neighbours. This average cannot be too close to three, because the large avalanches would cause many grains to spill over the edge, significantly decreasing the average size of piles across the lattice. It also cannot be too small, for small average pile sizes I would expect the average pile size to steadily increase with very few avalanches, whereas in the stationary state the average pile size has to remain constant.

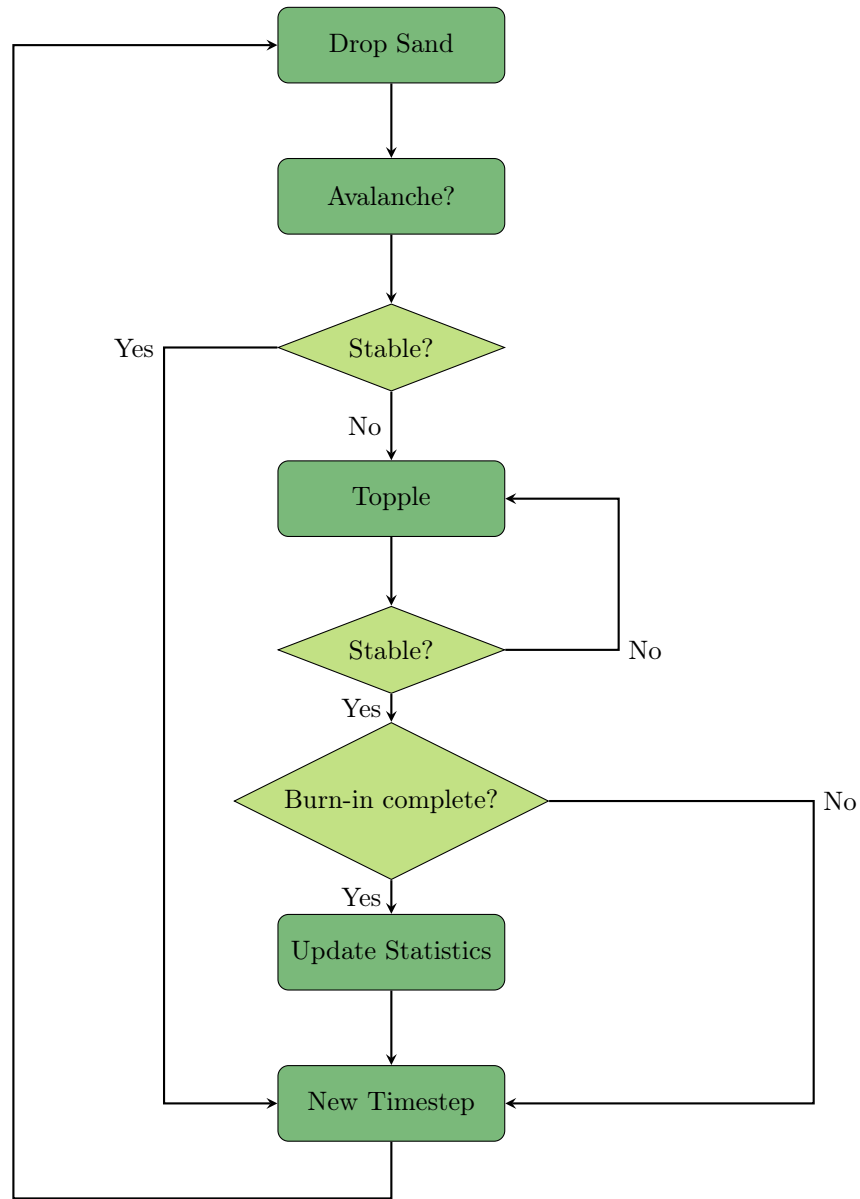
2 Methodology

2.1 Code Outline

The basic structure of my code for simulating the sandpile model was to initialise a lattice of zeroes, then every timestep, drop a grain of sand on a randomly chosen site from the grid. I then called my avalanche function, which repeatedly called the topple function for unstable sites to redistribute grains of sand to their neighbours until the grid was stable again. A flow chart is included on the next page to show the structure of the code.

2.2 The Topple and Avalanche Functions

The actual simulation of the lattice using the Topple and Avalanche functions were actually quite simple. Once the sand has been dropped, the avalanche function checks for unstable sites. If a site is found, it is added to a global set called "affected sites". While affected sites is non empty, a site is chosen from the set and, if the pile of sand at the site is at or above the threshold, the topple function is called for the site, meaning the pile at that site loses four of its grains of sand and each of its neighbours gains a grain of sand. Within the toppling function, these neighbours are also added to the set "affected sites". I had initially done this in the avalanche function, but this meant that I had to check neighbours twice for each topple, once in the topple function to check if a grain should be added, and once in the avalanche function to check if the neighbour should be added to affected sites. Doing this in the toppling function sped up my code by a few seconds. The neighbours may not be at or above the threshold, which is why each site from "affected sites" is checked before it is toppled. The set is used to avoid having to check the whole grid for unstable sites after each topple, of which there can be thousands per timestep. This method of toppling and avalanching is sufficient to simulate the lattice and there are actually faster methods of doing so using vectorisation. However, such methods make it difficult to keep track of the relevant statistics for each avalanche, specifically area and length as they complete multiple topples simultaneously and do not keep track of the specific sites that are toppled.



2.3 Recording Statistics

I recorded all of the statistics at the end of each timestep in the "[statistic]_history" array, and used default dictionaries "[statistic]_counts" to count the number of times each statistic recorded a certain value. I found the statistics using "curr_[statistic]", which was either updated progressively during the avalanche or topples, or was updated at the end of the avalanche.

Mass: `curr_mass` is incremented in the `drop_sand` function, then decreased by four in the `topple` function every time a topple occurs before being increased by one every time the `topple` function adds a grain of sand to a site. Normally, this results in the decrease in 4 being added back in its entirety, but if the topple occurs on the boundary of the lattice, some of the grains will not be added back with the decrease in the mass reflecting the loss of grains over the boundary.

Area: at the start of each avalanche a global set, `"toppled_sites"` is set to contain any sites that are unstable (which can only be one or zero initially). Before a site is toppled, it is added to `toppled_sites` within the `avalanche` function. At the end of the avalanche, `curr_area` is set to be the length of `toppled_sites`.

Topples: `curr_topples` is increased by one every time the `avalanche` function calls the `topple` function

Loss: at the end of the avalanche, `curr_loss` is defined as follows:

$$\text{curr_loss} = \text{mass_history}[\text{curr_time}-1] - \text{curr_mass} + 1,$$

with the extra 1 accounting for the fact that a grain of sand has been added during the timestep, and with `mass_history[curr_time-1]` being the mass of the grid at the previous timestep.

Length: length was the most complicated to calculate. For each site in `toppled_sites`, the Manhattan distance to the `initial_site` is calculated. If the site is not a corner of the lattice, the distance associated with this site is set to be one more than the Manhattan distance, since at least one of the neighbours of this site will have a Manhattan distance from `initial_site` that is further. If the site is a corner of the lattice, this is not the case, so the distance associated with this site is set to be the Manhattan distance from this site to `initial_site`. I appended each of these distances to a list, `"lengths"` and then set `curr_length` to be the maximum value in `lengths`. To determine whether sites were corners, I initialised a list of corner sites in the `__init__()` function for the class and checked whether the site is in that list.

Time Average: I initialised a grid of zeroes, then in the `run()` function, after each timestep I added the current grid to `space_average_history`, then at the end divided by the number of at the end to get the average pile size at each site over time.

Space Average History: I initialised an array of zeroes with the same length as the number of timesteps, then set `space_average_history[curr_] time` to be the mass of the grid divided by the number of sites in the grid.

2.4 Burn-In

I defined the functions "find_stats" and "update_stats" to find the statistics at the end of each avalanche and add them to the counts dictionaries only once the burn-in period was finished, ($\text{curr_time} > \text{burn_in}$) so that I did not have to use time calculating statistics that I did not need. To determine what the burn-in should be, I ran my code to make lattices for square grids with side lengths 20, 40, 60, 80 and 100 and plotted the `space_average_history` over time for each of them to determine roughly where it begins to plateau. These lattices were overall smaller than the ones I would eventually analyse because I wanted to find an estimate for the burn-in period before running the code on large lattices to save time. As seen from these plots, the point where it begins to plateau was quite easy to see by inspection. I then plotted the timesteps needed to plateau vs the area of the grid, which was linear as shown below and found the linear regression formula (with y intercept set to 0) to find the constant of proportionality. I then set `burn_in` to be this constant times the number of sites in the lattice. This may not work as well for rectangular lattices, which can be simulated using my code, but my analysis focuses on square lattices.

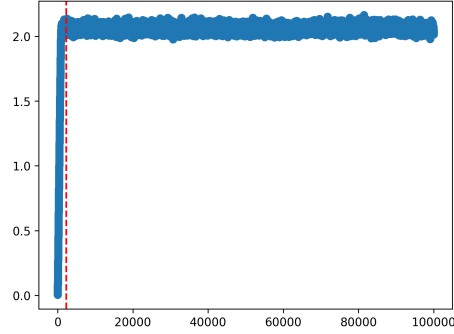


Figure 1: Space Average over Time for 20x20 Lattice

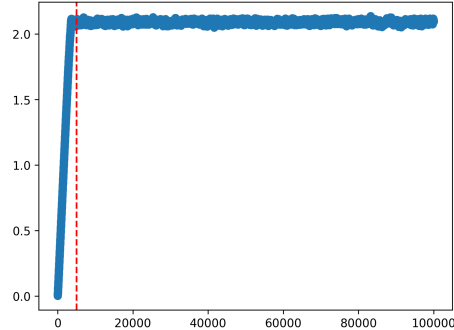


Figure 2: Space Average over Time for 40x40 Lattice

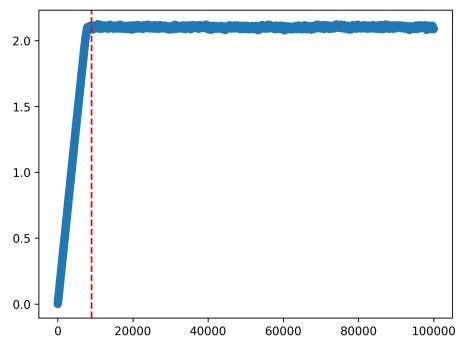


Figure 3: Space Average over Time for 60x60 Lattice

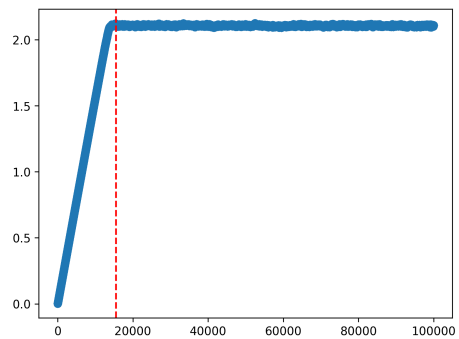


Figure 4: Space Average over Time for 80x80 Lattice

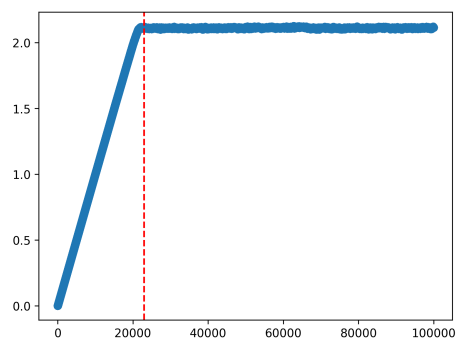


Figure 5: Space Average over Time for 100x100 Lattice

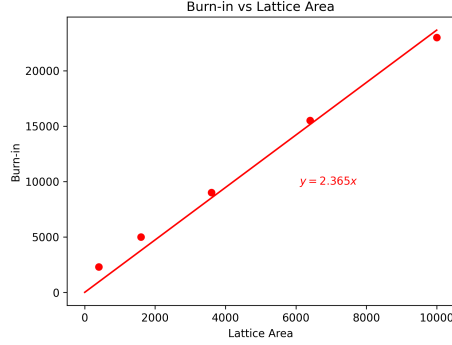


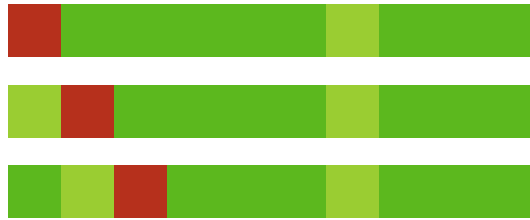
Figure 6: Burn In Coefficient

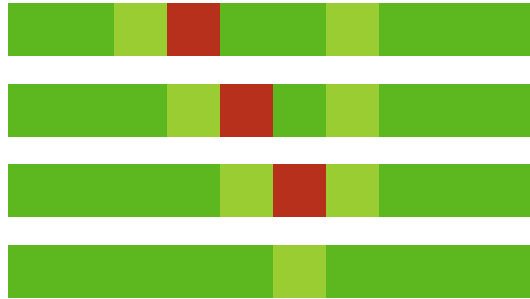
3 Results

3.1 Ergodicity

Figures 1-5 indicate that the system reaches a statistically stationary state after the burn-in period, because although the average pile size does change over time, its mean and variance remain the same over time. However, we can actually go further than this and conclude that the system is ergodic. A one dimensional example found in Redig [5] is informative on this topic. Imagine there is a line of writers in a newsroom and each worker can work on a maximum of one article at a time. Each hour, their boss enters the room with a new article to work on and gives it to one of the writers, selected uniformly at random. If they have more than one article to work on, they distribute an article to each of their two neighbours, leaving them with zero articles. The furthest right and furthest left writers have bins next to their desk, and if they have more than one article to work on, they pass an article to their one neighbour and dispose of the other article.

Once there are $n - 1$ articles being worked on by the n writers, at each timestep either: the one writer who does not have an article will receive an article to work on from the boss, or another writer will, causing an "avalanche" of articles and resulting in redistribution of the articles. Giving an article to the furthest right writer one there are $n - 1$ articles being worked on (with the furthest right writer working on one already) results in the following avalanche:





Starting the avalanche elsewhere means that the writer without an article differs:



Imagine there are mirrors located half way in between the writer with no articles and the edge, then we see that, once the system stabilises, the writer who ends up with no articles will be located at the mirror image of the site at which the first article was added:

Initial Unstable Configuration:



Final Stable Configuration:



Note that I omitted the imaginary mirror on the right which would govern where the writer with no articles would be post stabilisation if the article was given to a writer to the right of the writer who starts with no articles. This means that, provided the system is in the recurrent state, it will stabilise to a different distribution for each writer that the next article is given to. This creates an irreducible, aperiodic Discrete Time Markov Chain for the recurrent states (stable states with $n - 1$ or n articles in them) with transition probabilities $\frac{1}{n}$, meaning the system is ergodic.

This ergodicity can be extended to the two dimensional model by considering the burning algorithm developed by Dhar [3]. The idea is, given a stable grid, sites are "burnt" if their pile size is greater than or equal to the number of neighbours they have in the lattice (initially this will only consist of sites on the boundary). Once these sites are removed, we are left with a new set of sites and the process is repeated. Due to the previous burning, some of the sites will now have less neighbours than their pile size (some of their neighbours have been burnt) and will therefore be burnt. This process is repeated until the set of sites not burnt is empty or there are no sites which can be burnt. If there are sites remaining once the process is complete, the state of the grid is transient, but if there are no sites remaining, it is recurrent. This concept can be used to create a bijection between the recurrent sites and a rooted spanning tree of the sites in the lattice [5]. In one dimension, a root vertex $*$ is added and it is connected to the vertices 1 and n , corresponding to the leftmost and rightmost sites/writers. Vertices in the graph are connected to a vertex which has been burned in the previous timestep. The choice of vertex is determined on some system of preferencing to ensure a bijection. The bijection can be extended to more dimensions, meaning that the number of recurrent states is tractable (although very difficult to calculate). The burning algorithm is also informative on the nature of recurrent states. For example, there cannot be two piles of size zero which are neighbours because these sites would never be burnt according to the algorithm. Similarly, a square of sites with piles of size 1 (or less) cannot exist because these sites will always have two neighbours and will not be burnt.

3.2 Fitting the Power Law

To test whether the different statistics follow the power laws expected for self organised criticality I used the counts dictionaries to plot log-log plots of the the number of occurrences of a particular value of a statistic (after the burn-in period) vs the value of the statistic. Using the log-log plots, it was evident that area, length and topples all followed power law, while loss did not as the log-log plot did not look linear.

I investigated how the power law changed for different sized lattices using square lattices with side lengths 80, 90, 100, 110 and 120. The tables below show the power laws that I obtained for each statistic and size of lattice. Calculating this power law proved to be quite a challenging process as the plots, look less linear for the largest values of $\log(\text{statistic})$, since the number of occurrences of

larger values of statistics tend to be close to zero and there are not enough occurrences of these for them to be likely to have converged to near their expected value. It was clear that I should discard some of these to enable the linear fit to better reflect those data points which I thought were more likely to be well estimated from the model. However, deciding how to discard these and how many was not simple. I wanted to keep my method consistent for each grid area, so I (with some help from AI) investigated three potential methods to discard values.

The simplest method was just to discard some proportion of the values at the end when fitting the linear model. I experimented with different proportions and how they affected the final plots. The advantage of discarding more values was that these values represented more avalanches from my simulation and since the probability of occurrences of smaller statistics (e.g smaller area) was higher, I expected that these values would be better estimated in the model. However, I did not want to ignore all the extreme values. After trying different proportions of both the largest area/length/topples (i.e. the number of keys in the dictionary) and different proportions of the grid area, I found that taking $\frac{1}{300}$ th of the grid area gave consistently good fits for all the models. Taking a proportion of the largest area/length/topples tended to give a better fit either for smaller grids or larger grids depending on the proportion.

Another method I tried was using `piecewise_regression` function in python. Since the drop off at the end itself looked fairly linear, I thought that `piecewise_regression` might be able to split the plots into two parts, and I could take the linear regression law for the first part. However, for larger lattices, the plots were more consistently linear and `piecewise_regression` failed to converge and split the plots into two parts, while for smaller lattices, the drop off point was not clear enough and `piecewise_regression` often fit a line which included too many extreme values worsening the fit for the lower values which I believed were better estimated.

The final method I tried was using `ransac_regressor` to create a mask of values which it determined did not fit as well as the others, and then omit these values. This was sometimes quite effective, however, because the function tests points at random to discard, sometimes when the function chose too many points from extreme values of the particular statistic, the fits were way off. Additionally, it did not reflect the fact that I wanted to discard larger values for the statistics.

Due to the constraints of the latter two methods, along with their increased time needs, I decided to use the first, simpler method to find the fit for each statistic and lattice. The plots for the 100 by 100 lattice are displayed below. The red lines of best fit were calculated excluding some of the extremely large areas (lengths/topples) but all values are shown on the scatter plot. I also included the a plot for of the loss vs the log of the number of occurrences as the log-log plot for loss was certainly not linear. The plot included however, though closer to being linear, was still not really linear so I did not fit regression to it.

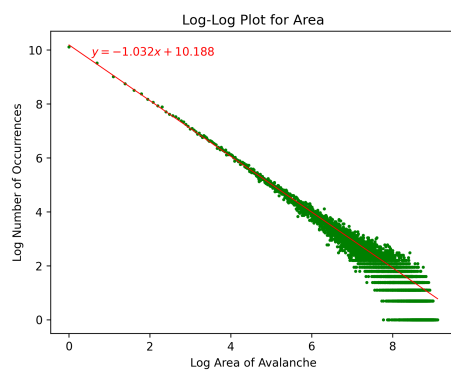


Figure 7: Log-Log Plot for Area

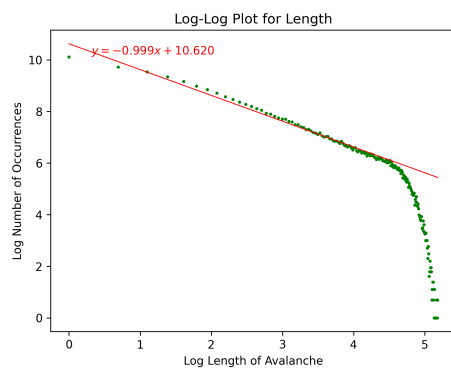


Figure 8: Log-Log Plot for Length

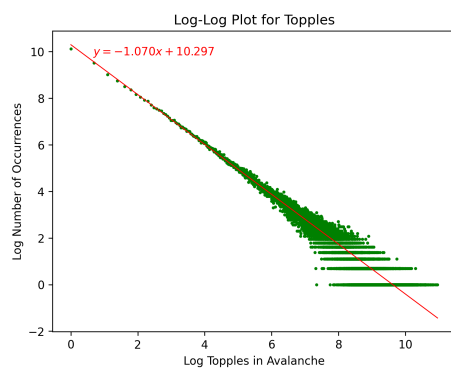


Figure 9: Log-Log Plot for Topples

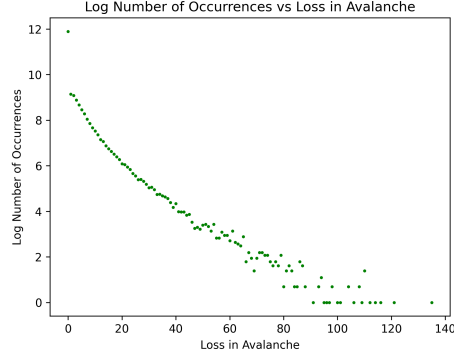


Figure 10: Log Plot for Loss

The exponents I obtained for each statistic across the different sized lattices are displayed in the table below.

Side Length	Area	Length	Topples
80	-1.042	-0.999	-1.054
90	-1.049	-1.000	-1.064
100	-1.032	-0.999	-1.064
110	-1.038	-1.021	-1.015
120	-1.043	-1.072	-1.049

Table 1: Exponents for the Distribution of Statistics across Varying Grid Sizes

The exponents for each of these statistics are close to negative 1 (within 10%). This matches the theoretical application of BTW models in explaining $\frac{1}{f}$ noise. Aside from the exponents for length, which become much more negative for larger grid areas, there is fluctuation in the exponents among the different grid sizes, indicating that the exponents are fairly consistent among different grid sizes, although more timesteps would be needed in the simulation to confirm this assumption. The exponents for length did get larger in magnitude for larger grids as the power law was more difficult to fit. Perhaps a different method of calculating length, such as the greatest distance between two affected (sites which were either toppled or received a grain (or grains) from a toppled site) sites would have been better. This would be more like a diameter and might be more consistent as it would not be as affected by the placement of the first site to avalanche. However, the exponents did remain quite close to -1 , and the fact that these key statistics of area, topples and length seem to be governed by power laws with exponents close to -1 indicates that the system displays self-organised criticality. This exponent of -1 corresponds to the $\frac{1}{f}$ noise that the sandpile model was developed to explain.

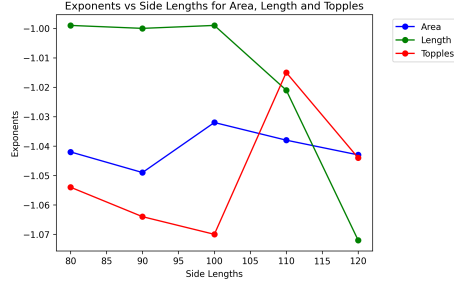


Figure 11: Exponents for Different Sized Lattices

3.3 Correlation

I expected significant positive correlation between topples and area, since topples is at least as big as area (and in many cases the same) and more likely to be larger than area if area is large. I also expected positive correlation between topples and length and area and length, although maybe not as much. I did expect some positive correlation between the three statistics and loss, but I was unsure how strong the correlation would be, since the loss would also depend on the site that triggers the avalanche; if this initial site is close to the edge of the lattices, it would be more likely that loss will occur in the resulting avalanche. But in this case the area and number of topples from the avalanche would be constrained as the spread would be constrained in at least one direction.

After simulating the 100 by 100 grid, I plotted correlation plots for each pair of statistics to see how they related to one another. The plots, displayed below, show interesting relationships between the statistics. For the correlation between length and area I plotted the square root of the area against the length since the correlation plot for length and area appeared to show a quadratic relationship.

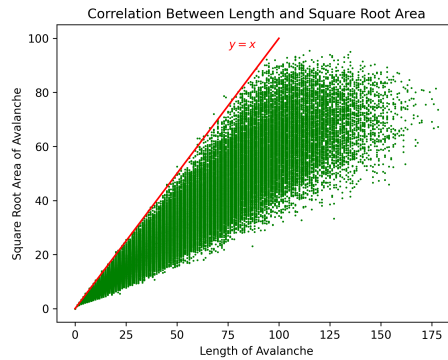
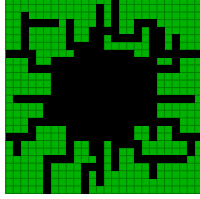


Figure 12: Square Root Area vs Length of Avalanche

This plot supports the contention that there is a quadratic relationship between the length and the area of the avalanche, although the fan shape indicates heteroskedasticity, with the variance away from this linear relationship dependent on the magnitude of the observations. This is probably because the length measurement is taken from the initial site, which is not necessarily centered, especially for large avalanches. If the initial site is not near the centre of the sites affected by the resulting avalanche, as might be the case if the avalanche propagates from a site near the edge of the lattice, the length will be large compared to the size of the avalanche as the length measurement will be closer to a diameter than a radius. The length is typically longer than the square root of the area, which makes sense since the Manhattan distance, not the Euclidean distance was used. The fact that length is typically larger than square root area could also be an indication of the shape of avalanches, tending to extend out in branches as shown in the diagram below:



The black lines represent affected sites, and it's easy to see from the diagram how the square root of the area can be smaller than the length due to the amount of unaffected sites (represented by the green squares) within the grid; even though the length of this avalanche would be 21, the area would be much less than $21^2 = 441$. Note that this is not an example from an actual avalanche but is included as an example of how the shape of an avalanche can lead to the length being much greater than the square root of the area of the avalanche.

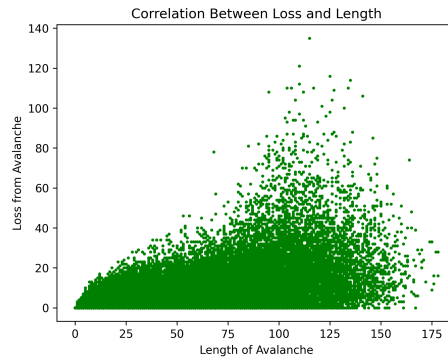


Figure 13: Loss vs Length of Avalanche

The plot shows that while there is more scope for higher loss from avalanches with a greater length, there are also many avalanches with very few grains of

sand lost across all lengths.

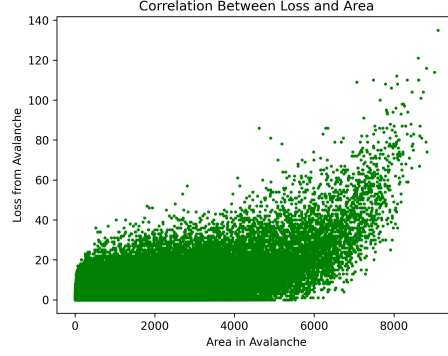


Figure 14: Loss vs Area of Avalanche

This plot is similar to the plot for loss and length, showing many avalanches with smaller loss across different areas of avalanche, although the correlation between the two is stronger than in the previous plot with most of the avalanches with the most loss also having a larger area. The ability for the loss of an avalanche to be extremely spread out among different lengths and areas of avalanche might be one of the reasons that the power law did not fit for loss. Other plots, such as the one comparing square root area or and length or the one comparing area and topples, showed significant correlation between the pairs of statistics. However, the loss plots tend to show potential for no or very little loss even when an avalanche is long (150 sites) or has a large area (6,000). The fact that the correlation is much clearer for large areas of avalanche suggests that perhaps the power law would be more easily fit in a larger grid where there might be more frequent large areas of avalanches. The large areas of avalanches follow a power law, so their corresponding large losses might also do so.

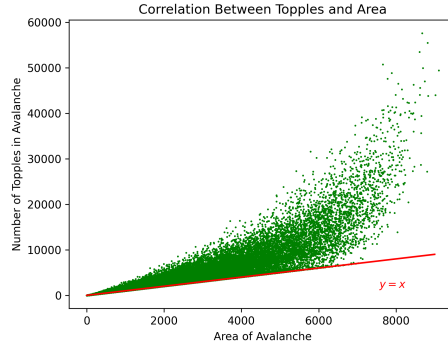


Figure 15: Topples vs Area of Avalanche

As predicted, there are many avalanches for which the topples and area are the same or close to the same, although for avalanches with the largest area the number of topples for the avalanche is much larger, even almost 8 times larger than the area of the avalanche. Of course, the more sites topples, the more likely sites are to topple multiple times as grains of sand are transferred between interior sites over the course of many topples.

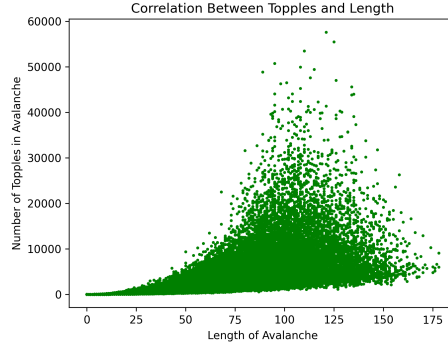


Figure 16: Topples vs Length of Avalanche

The mountain shape of the correlation plot for topples and length reflects the fact that avalanches beginning near the centre are more likely to involve more topples, since grains can transfer between sites over many topples before the propagations reach the edge of the lattice and grains are lost. Avalanches with lengths between 150 and 200 have to start closer to the edges of the lattice for this length measurement to be possible, meaning the number of topples is limited by the position of the first site to be toppled.

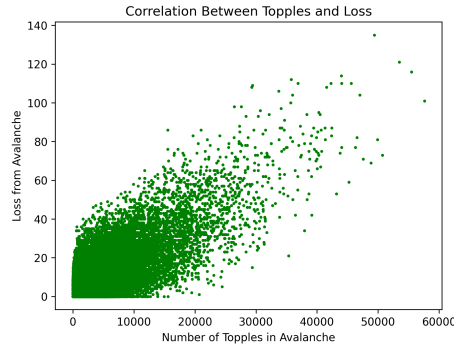


Figure 17: Topples vs Loss of Avalanche

This plot seems to be linear, with even the variance apparently consistent and unaffected by the magnitude of the topples and/or loss. This suggests that

whenever there are a lot of topples, these propagate to the edges of the lattice and cause significant loss of grains of sand over the edge, with this loss linearly proportional to the number of topples. It is interesting that this correlation is much stronger than the correlation between loss and length or area. This shows what is seen in the

4 Conclusion

4.1 Summary of Findings

The results in Table 1 and Figure 11 are evidence of the self-organised criticality of the system because they show that key statistics for the system, area, length and topples follow a power law with exponent close to -1 , independent of the size of the lattice (or at least, independent in the case of topples and area). The loss of each avalanche was more difficult to explain theoretically, perhaps due to the size of the grid.

4.2 Potential Extensions

This project could be extended in many ways, although for the purposes of simulation it is important to maintain the Abelian property of avalanches. Dropping grains of sand at the same site forms fractal structures and could lead to interesting analysis on the distributions of key statistics with the system now being deterministic (although still very complicate). The system can also be designed to include sink sites which are always zero and act like holes for the grains of sand to slip through. The effect of such sinks on the statistics of avalanches could then be studied, specifically the impact of increasing the proportion of sinks or changing the geometry of the placement of the sinks. Dhar et. al [2] also extended the sandpile model to construct scalar functions which are linear in height variables of the pile, proving them to also invariant under toppling at a given site which enable the recurrent configurations to be labelled. This could then be used to simulate the transitions between recurrent states themselves and confirm that each state occurs uniformly at random, as is the case with the one dimensional model. The concept of self-organised criticality itself is also be useful in analysis many different stochastic models on lattices, such as bond percolation on lattices [4], where a complete lattice is taken and then each edge is kept with probability $p \in (0, 1)$ independent of the other edges. Such models can also be simulated and statistics such as the maximum size of a cluster of connected vertices analysed in a similar manner to what was done in this project. Such simulation is a vital tool in analysing all stochastic models and enabling their corresponding real world phenomena to be understood in greater detail.

References

- [1] Per Bak, Chao Tang, and Kurt Wiesenfeld. Self-organized criticality: An explanation of the $1/f$ noise. *Phys. Rev. Lett.*, 59:381–384, Jul 1987.
- [2] D. Dhar, P. Ruelle, S. Sen, and D. N. Verma. Algebraic aspects of abelian sandpile models, 1994.
- [3] Deepak Dhar. The abelian sandpile and related models. *Physica A: Statistical Mechanics and its Applications*, 263(1-4):4–25, February 1999.
- [4] Antal A. Járai. Sandpile models. *Probability Surveys*, 15(none):243 – 306, 2018.
- [5] Frank Redig. Mathematical aspects of the abelian sandpile model. 2005.